

SQL NULL Functions

11 · NULL Functions — Complete Reference Guide

Table of Contents

1. Handle NULL — Data Aggregation
2. Handle NULL — Mathematical Operators
3. Handle NULL — Sorting Data
4. NULLIF — Division by Zero
5. IS NULL / IS NOT NULL
6. LEFT ANTI JOIN
7. NULLs vs Empty String vs Blank Spaces

1. Handle NULL — Data Aggregation

→ *Task 1: Average scores — COALESCE replaces NULL Score with 0*

```
SELECT
    CustomerID,
    Score,
    COALESCE(Score, 0) AS Score2,
    AVG(Score) OVER () AS AvgScores,
    AVG(COALESCE(Score,0)) OVER () AS AvgScores2
FROM Sales.Customers;
```

2. Handle NULL — Mathematical Operators

→ *Task 2: Full name concatenation and score with bonus — handle NULLs*

```
SELECT
    CustomerID,
    FirstName,
    LastName,
    FirstName + ' ' + COALESCE(LastName, '') AS FullName,
    Score,
```

```
COALESCE(Score, 0) + 10
FROM Sales.Customers;
```

AS ScoreWithBonus

3. Handle NULL — Sorting Data

→ Task 3: Sort customers by score (lowest first), NULLs appear last

```
SELECT
    CustomerID,
    Score
FROM Sales.Customers
ORDER BY
    CASE WHEN Score IS NULL THEN 1 ELSE 0 END,
    Score;
```

4. NULLIF() — Division by Zero

→ Task 4: Sales price per order — NULLIF prevents division by zero

```
SELECT
    OrderID,
    Sales,
    Quantity,
    Sales / NULLIF(Quantity, 0) AS Price
FROM Sales.Orders;
```

5. IS NULL / IS NOT NULL

→ Task 5: Customers with no score

```
SELECT *
FROM Sales.Customers
WHERE Score IS NULL;
```

→ Task 6: Customers who have a score

```
SELECT *
FROM Sales.Customers
WHERE Score IS NOT NULL;
```

6. LEFT ANTI JOIN — Using NULL Check

→ Task 7: Customers who have NOT placed any orders

```
SELECT
    c.*,
    o.OrderID
FROM Sales.Customers AS c
```

```
LEFT JOIN Sales.Orders AS o
  ON c.CustomerID = o.CustomerID
WHERE o.CustomerID IS NULL;
```

7. NULLs vs Empty String vs Blank Spaces

→ *Task 8: Demonstrate differences between NULL, empty string, and blank spaces*

```
WITH Orders AS (
  SELECT 1 AS Id, 'A' AS Category UNION
  SELECT 2, NULL UNION
  SELECT 3, '' UNION
  SELECT 4, ' '
)
SELECT
  *,
  DATALENGTH(Category) AS LenCategory,
  TRIM(Category) AS Policy1,
  NULLIF(TRIM(Category), '') AS Policy2,
  COALESCE(NULLIF(TRIM(Category), ''), 'unknown') AS Policy3
FROM Orders;
```